

3 Sum

Hectron

July 28, 2026

Contents

1	Description	2
2	Function/Method Declaration	2
3	Solution	2
3.1	First pairs	2
3.2	Other pairs	3
3.3	FlowChart	3

1 Description

Given an array `nums` of `n` integers, return an array of **all the unique quadruplets**: `[nums[a], nums[b], nums[c]]` such that:

- $0 \leq a, b, c, d < n$
- `a`, `b`, `c`, and `d` are distinct.
- `nums[a] + nums[b] + nums[c] + nums[d] == target`

2 Function/Method Declaration

```
std::vector< std::vector<int> > threeSums(std::vector numbers, int target);
```

3 Solution

- I. Mix **numbers** with itself to find all pair combinations.
- II. Recent sum pairs produced are crossed with **numbers** to produce another list of pairs.
- III. II is repeated `N-2` times (i.e. `N=4`; 2 iterations)

3.1 First pairs

Method below is used to find unique pairs of two equal vectors (i.e. autocorrelation samples).

Raw vector: `numbers = [ 0, -4, 3, 1, 0 ]`

Sort set: `numbers = [ -4, 0, 0, 3, 3 ]`

Unquie values in vector: `numbers = [ -4, 0, 1, 3 ]`

Pairs (diagonal search):

Table 1: Sums				
List	-4	0	1	3
-4	-8	-4	-3	-1
0	-4	0	1	3
1	3	1	2	4
3	-1	3	4	3

3.2 Other pairs

Method below is used to find unique pairs of input vector with previous sum pairs:

Input set: numbers = [-4, 0, 1, 3]

Sums set: numbers = [-8, -4, -1, 0, 1, 2, 3, 4]

Table 2: Sums				
List	-4	0	1	3
-8	-12	-8	-7	-5
-4	-8	-4	-3	-1
-1	-5	-1	0	2
0	-4	0	1	3
1	-3	1	2	4
2	-2	2	3	5
3	-1	3	4	6
4	0	4	5	7

3.3 FlowChart

